

竞赛 API 背诵清单

适用范围：ICPC / CCPC / 蓝桥杯 / 团体程序设计天梯赛，默认主语言为 C++。如果你用 Java 或 Python，本文末尾给出只需要额外记住的高频 API。

更新时间：2026-04-27。比赛规则每年、每站可能不同，赛前必须看当年正式通知。

三份文档的分工：

- 本文解决“库函数和语言 API 怎么写”的问题。
- [竞赛算法板子背诵清单.md](#) 解决“哪些算法模板要背、什么题型用”的问题。
- [竞赛实战技巧与调试对拍清单.md](#) 解决“怎么写得稳、怎么调试、怎么对拍”的问题。

1. 比赛中能不能看资料

结论：不要假设“比赛时一定能查文档”。训练时按“完全不能查资料”准备最稳。

比赛	常见资料限制	备赛建议
ICPC	规则强依赖赛区和赛季。很多现场赛只允许纸质 Team Reference Document，常见限制是 25 页 A4/Letter；不允许电子资料、U 盘、手机等。World Finals 近年会在比赛环境提供有限在线参考资料，如 C++ STL 文档、JDK JavaDocs，同时队伍参考文档按规定提交/打印。部分线上区域赛可能允许公开在线资源。	STL 基础 API 必背；算法模板可整理进 25 页队伍文档；赛前核对本赛区 TRD 页数、是否单面、是否可手写标注。
CCPC	公开总决赛规则曾允许携带书籍、字典、手册、程序清单等文字性参考资料，但禁止可计算机处理的电子设备和通讯工具。分站/年份可能有细则差异。	仍按不能查电子资料准备；常用模板可打印；别依赖 IDE 自动补全。
蓝桥杯	个人赛软件类通常是封闭限时，答题中不允许访问互联网，也不允许使用本机以外资源；比赛管理办法还写明不得携带书籍、笔记、资料、通讯及存储设备等。部分环境说明会提供 C/C++ API 帮助文档。	必背常用 STL 和库函数；不要指望能翻自己资料；本地帮助文档可用与否以当届环境为准。
天梯赛 GPLT	公开规程常见要求是仅可携带无计算功能的铅笔或水笔入场，不得携带可计算机处理的软件/数据、存储设备、计算器、通讯工具。线上/线下规则也可能禁止打开预存电子资料和访问网站。	当成完全闭卷；API、输入输出、字符串、排序、容器必须熟。

常用在线文档：

- C++: <https://cppreference.com/>、<https://cplusplus.com/reference/>
- Java: <https://docs.oracle.com/en/java/javase/>
- Python: <https://docs.python.org/3/>
- 算法知识: <https://oi-wiki.org/>、<https://cp-algorithms.com/>
- 比赛规则入口: ICPC <https://icpc.global/>，CCPC <https://ccpc.io/>，蓝桥杯 <https://dasai.lanqiao.cn/>，天梯赛 <https://gplt.patest.cn/regulation>

赛前环境检查：

- 确认 C++ 标准。ICPC World Finals 2025 环境说明使用 `g++ 13.2.0` 和 `-std=gnu++20`；ICPC EUC 2026 计划环境使用 `g++ 14.2.0` 和 `-std=c++23`。但国内区域赛、蓝桥杯、天梯赛可能仍以 C++14/17 或本地 IDE 为准。
- 不确定时按 C++17 写最稳：`structured binding`、`gcd/lcm`、`optional` 这类可以用；`ranges`、`std::span`、`std::format` 不建议作为默认依赖。
- 赛前确认栈大小、是否静态链接、是否支持 `bits/stdc++.h`、评测机语言版本、文件扩展名要求。
- 训练时不要依赖 IDE 自动补全；常用函数名、头文件、比较器写法要能手写。

2. C++ 头文件与基础模板

ICPC/CCPC/Linux 环境常用：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
const int INF = 0x3f3f3f3f;
const ll LINF = (1LL << 62);

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    return 0;
}
```

蓝桥杯 Dev-C++ / Windows 一般也支持 `bits/stdc++.h`，但更稳的写法是记住核心头：

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <queue>
#include <stack>
#include <map>
#include <set>
#include <unordered_map>
#include <unordered_set>
#include <cmath>
#include <cstring>
#include <numeric>
#include <iomanip>
#include <sstream>
#include <array>
#include <chrono>
#include <limits>
#include <stdint>
#include <cctype>
#include <functional>
```

3. 输入输出 API

```
int x; long long y; string s;
cin >> x >> y >> s;           // 按空白分割
getline(cin, s);                // 读整行

cout << fixed << setprecision(10) << ans << '\n';
```

常见坑：

```
int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
getline(cin, s);
```

`cin >> n` 后如果要 `getline`，必须吃掉行尾换行。

读到 EOF：

```
while (cin >> x) {
    // use x
}
```

整行解析，天梯赛和模拟题很常见：

```
string line;
getline(cin, line);
stringstream ss(line);
while (ss >> x) {
    // use x
}
```

C 风格高性能输入输出：

```
scanf("%d%lld", &x, &y);
printf("%.10f\n", ans);
```

4. vector

```
vector<int> a;           // 空
vector<int> b(n);        // n 个 0
vector<int> c(n, -1);    // n 个 -1

a.reserve(n);
a.push_back(x);
a.emplace_back(x);
a.pop_back();
a.size();
a.empty();
a.clear();
a.resize(n);
a.assign(n, 0);
a.front();
a.back();
a[i];
```

```
sort(a.begin(), a.end());
reverse(a.begin(), a.end());
```

注意: `a.size()` 返回无符号类型, 和 `int` 混用时容易出警告或下溢。循环倒着写时推荐先转成 `int n = (int)a.size();`。

删除与去重:

```
a.erase(a.begin() + i);          // 删除下标 i, O(n)
a.erase(a.begin() + l, a.begin() + r); // 删除 [l, r)

sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
```

遍历时删除:

```
for (auto it = a.begin(); it != a.end(); ) {
    if (*it == x) it = a.erase(it);
    else ++it;
}
```

二维数组:

```
vector<vector<int>>> g(n, vector<int>(m, 0));
```

固定长度数组:

```
array<int, 4> dx = {-1, 0, 1, 0};
array<int, 4> dy = {0, 1, 0, -1};
```

`array` 长度是编译期常量, 适合方向数组、小矩阵、小状态; 需要运行期长度时用 `vector`。

5. string

```
string s = "abc";
s.size();
s.empty();
s.push_back('x');
s.pop_back();
s += "def";
s[i];
s.substr(pos, len);          // 从 pos 起 len 个
s.find("ab");                // 找不到返回 string::npos
reverse(s.begin(), s.end());
sort(s.begin(), s.end());
```

字符串与数字:

```
int x = stoi(s);
long long y = stoll(s);
double z = stod(s);
string t = to_string(x);
```

字符判断与转换：

```
isdigit(c);
isalpha(c);
islower(c);
isupper(c);
tolower(c);
toupper(c);
```

注意：`isdigit` 等函数建议写成 `isdigit((unsigned char)c)`，避免负 `char` 出问题。

字符串分割：

```
stringstream ss(s);
string part;
while (getline(ss, part, ',')) {
    // comma separated part
}
```

`s.find()` 返回的是 `size_t`，找不到时是 `string::npos`。不要把结果直接存进 `int` 后再和 `npos` 比较。

6. pair / tuple

```
pair<int, int> p = {x, y};
p.first;
p.second;

vector<pair<int, int>> v;
sort(v.begin(), v.end()); // 先 first, 再 second

tuple<int, int, string> t = {1, 2, "x"};
auto [a, b, c] = t; // C++17
```

自定义排序：

```
sort(v.begin(), v.end(), [](const pii& a, const pii& b) {
    if (a.first != b.first) return a.first < b.first;
    return a.second > b.second;
});
```

7. queue / stack / deque

```
queue<int> q;
q.push(x);
q.front();
q.pop();
q.empty();
q.size();

stack<int> st;
st.push(x);
st.top();
st.pop();
```

```

deque<int> dq;
dq.push_front(x);
dq.push_back(x);
dq.pop_front();
dq.pop_back();
dq.front();
dq.back();

```

单调队列常用 `deque<int>` 存下标。

8. priority_queue

默认大根堆：

```

priority_queue<int> pq;
pq.push(x);
pq.top();
pq.pop();

```

小根堆：

```

priority_queue<int, vector<int>, greater<int>> pq;

```

pair 小根堆：

```

priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;

```

自定义比较：

```

struct Node { int x, y; };
struct Cmp {
    bool operator()(const Node& a, const Node& b) const {
        return a.x > b.x;    // x 小的优先
    }
};
priority_queue<Node, vector<Node>, Cmp> pq;

```

注意：`priority_queue` 的比较器和 `sort` 直觉相反，返回 `true` 表示优先级更低。

9. set / map

```

set<int> s;
s.insert(x);
s.erase(x);
s.count(x);           // 0 或 1
s.find(x);
*s.begin();           // 最小
*s.rbegin();          // 最大
s.lower_bound(x);     // 第一个 >= x
s.upper_bound(x);     // 第一个 > x

```

```
map<string, int> mp;
mp[key]++;           // 不存在会自动插入 0
mp.count(key);
mp.find(key);
mp.erase(key);

for (auto [k, v] : mp) {}
```

找前驱后继:

```
auto it = s.lower_bound(x);
if (it != s.end()) {
    int ge = *it;           // >= x
}
if (it != s.begin()) {
    --it;
    int lt = *it;           // < x
}
```

multiset:

```
multiset<int> ms;
ms.insert(x);
ms.erase(ms.find(x));      // 只删一个, 前提是存在
ms.erase(x);               // 删除所有等于 x 的元素
```

遍历时删除 set/map 元素:

```
for (auto it = s.begin(); it != s.end(); ) {
    if (bad(*it)) it = s.erase(it);
    else ++it;
}
```

10. unordered_map / unordered_set

```
unordered_map<string, int> mp;
unordered_set<int> st;

mp.reserve(n * 2);
mp.max_load_factor(0.7);
```

适合平均 O(1) 查询。遇到卡哈希或需要有序遍历, 用 map/set。

整数键防卡哈希:

```
struct CustomHash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t seed =
```

```

        chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + seed);
    }
};
unordered_map<long long, int, CustomHash> safe_mp;

```

自定义 pair 哈希:

```

struct PairHash {
    size_t operator()(const pair<int,int>& p) const {
        return ((uint64_t)(uint32_t)p.first << 32) ^ (uint32_t)p.second;
    }
};
unordered_map<pair<int,int>, int, PairHash> mp;

```

11. algorithm

排序:

```

sort(a.begin(), a.end());
sort(a.rbegin(), a.rend());
stable_sort(a.begin(), a.end());

```

最值:

```

min(x, y);
max(x, y);
min({a, b, c});
max({a, b, c});
*min_element(a.begin(), a.end());
*max_element(a.begin(), a.end());

```

二分:

```

auto it1 = lower_bound(a.begin(), a.end(), x); // >= x
auto it2 = upper_bound(a.begin(), a.end(), x); // > x
int pos = lower_bound(a.begin(), a.end(), x) - a.begin();
bool ok = binary_search(a.begin(), a.end(), x);

```

排列:

```

sort(a.begin(), a.end());
do {
    // use a
} while (next_permutation(a.begin(), a.end()));

```

其他:


```
reverse(a.begin(), a.end());
rotate(a.begin(), a.begin() + k, a.end());
fill(a.begin(), a.end(), 0);
copy(a.begin(), a.end(), b.begin());
count(a.begin(), a.end(), x);
find(a.begin(), a.end(), x);
nth_element(a.begin(), a.begin() + k, a.end()); // 第 k 小归位, 左侧不保证有序
all_of(a.begin(), a.end(), [](int x) { return x > 0; });
any_of(a.begin(), a.end(), [](int x) { return x == 0; });
```

12. numeric

```
long long sum = accumulate(a.begin(), a.end(), 0LL);
int g = gcd(x, y); // C++17, <numeric>
int l = lcm(x, y); // C++17

vector<int> p(n);
iota(p.begin(), p.end(), 0); // 0,1,2,...
```

前缀和:

```
vector<long long> pre(n + 1);
for (int i = 0; i < n; i++) pre[i + 1] = pre[i] + a[i];
long long sum_l_r = pre[r + 1] - pre[l]; // [l, r]
```

13. cmath / 数学函数

```
abs(x); // 整数或浮点, 注意 long long 可用 llabs 或 std::abs
sqrt(x);
pow(a, b);
sin(x); cos(x); tan(x);
asin(x); acos(x); atan2(y, x);
floor(x); ceil(x); round(x);
log(x); log10(x); exp(x);
```

常量:

```
const double PI = acos(-1.0);
const double EPS = 1e-9;
```

浮点比较:

```
if (fabs(a - b) < EPS) {}
```

整数上取整:

```
long long ceil_div(long long a, long long b) {
    return (a + b - 1) / b; // 仅适用于 a,b 非负且 b>0
}
```

long long 乘法可能溢出时:

```

using i128 = __int128_t;

i128 v = (i128)a * b;

void print_i128(i128 x) {
    if (x < 0) {
        cout << '-';
        x = -x;
    }
    if (x >= 10) print_i128(x / 10);
    cout << char('0' + x % 10);
}

```

14. cstring / 数组初始化

```

int a[N];
memset(a, 0, sizeof a);
memset(a, -1, sizeof a);
memset(a, 0x3f, sizeof a);    // int INF

```

注意：memset 按字节填充，只适合 0、-1、0x3f 这类常见值。普通值用 fill：

```

fill(a, a + n, 7);
fill(&g[0][0], &g[0][0] + N * M, INF);

```

15. bitset

```

bitset<1000> b;
b.set();
b.reset();
b.flip();
b.count();
b.any();
b.none();
b[i] = 1;

bitset<1000> c = b << 3;
auto d = b & c;

```

常用于状态压缩、可达性、集合交并。

16. 常见容器复杂度

API	复杂度
<code>vector</code> 尾部 <code>push_back/pop_back</code>	均摊 $O(1)$
<code>vector</code> 中间插入/删除	$O(n)$
<code>sort</code>	$O(n \log n)$
<code>lower_bound</code> on <code>vector</code>	$O(\log n)$, 前提已排序
<code>set/map</code> 插入/删除/查找	$O(\log n)$
<code>unordered_map/set</code> 平均插入/删除/查找	$O(1)$
<code>priority_queue</code> <code>push/pop</code>	$O(\log n)$
<code>priority_queue</code> <code>top</code>	$O(1)$

17. 竞赛最常见模板 API

并查集：

```
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

取模：

```
const long long MOD = 1000000007;

long long mod_pow(long long a, long long b) {
    long long r = 1 % MOD;
    while (b) {
        if (b & 1) r = r * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return r;
}
```

Dijkstra 小根堆：

```
vector<vector<pair<int,int>>> g(n);
vector<long long> dist(n, LINF);
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>> pq;

dist[s] = 0;
```

```

pq.push({0, s});
while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();
    if (d != dist[u]) continue;
    for (auto [v, w] : g[u]) {
        if (dist[v] > d + w) {
            dist[v] = d + w;
            pq.push({dist[v], v});
        }
    }
}
}

```

BFS:

```

queue<int> q;
vector<int> dist(n, -1);
dist[s] = 0;
q.push(s);
while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int v : g[u]) {
        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}
}

```

18. 蓝桥杯常见额外 API

蓝桥杯常有日期、枚举、暴力模拟、精确格式输出。

```

// 枚举排列
vector<int> a = {1,2,3,4};
do {
} while (next_permutation(a.begin(), a.end()));

// 格式补零
cout << setw(2) << setfill('0') << x << '\n';

// long long 乘法前转型
long long area = 1LL * w * h;

```

日期判断:

```

bool leap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}
int mdays(int y, int m) {
    static int d[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
    if (m == 2 && leap(y)) return 29;
    return d[m];
}

```

19. Java 高频 API

```
Scanner sc = new Scanner(System.in);    // 慢但方便
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
StringTokenizer st = new StringTokenizer(br.readLine());

ArrayList<Integer> a = new ArrayList<>();
a.add(x); a.get(i); a.set(i, x); a.size();

Collections.sort(a);
Arrays.sort(arr);

HashMap<String, Integer> mp = new HashMap<>();
mp.put(k, mp.getOrDefault(k, 0) + 1);
mp.containsKey(k);

PriorityQueue<Integer> pq = new PriorityQueue<>();    // 小根堆
PriorityQueue<Integer> mx = new PriorityQueue<>(Collections.reverseOrder());
```

大整数:

```
BigInteger a = new BigInteger("123");
BigInteger b = new BigInteger("456");
a.add(b); a.subtract(b); a.multiply(b); a.divide(b); a.mod(b);
a.gcd(b); a.compareTo(b);
```

20. Python 高频 API

```
import sys, math, heapq, bisect
input = sys.stdin.readline

a = list(map(int, input().split()))
g = math.gcd(x, y)

heap = []
heapq.heappush(heap, x)
x = heapq.heappop(heap)

i = bisect.bisect_left(a, x)
j = bisect.bisect_right(a, x)

from collections import deque, Counter, defaultdict
q = deque([s])
q.append(x); q.popleft()
cnt = Counter(a)
d = defaultdict(int)
```

21. 必背优先级

第一优先级，必须不用想就能写：

- `vector/string/sort/lower_bound/upper_bound`
- `queue/stack/priority_queue`
- `set/map/unordered_map`

- gcd/accumulate/iota
- memset/fill
- fixed/setprecision
- next_permutation

第二优先级，常用但可稍慢回忆：

- deque 单调队列
- bitset
- tuple/structured binding
- substr/find/stoi/stoll
- multiset 删除一个元素
- 自定义排序和自定义哈希

第三优先级，建议放进纸质模板：

- 并查集
- 快速幂
- Dijkstra
- BFS/DFS 图模板
- 日期函数
- KMP、Trie、线段树、树状数组等算法模板

22. 提交前检查清单

每题提交前快速扫一遍：

- 是否清空了多组数据中的全局数组、图、队列、优先队列。
- 数组大小是否覆盖最大 n/m ，线段树是否开到 $4*n$ ，Trie/边数组是否按总量开。
- 距离、方案数、乘法是否需要 long long 或 __int128。
- 二分边界是求最小可行值还是最大可行值。
- 比较函数是否满足严格弱序，没有写成 $<=$ 。
- priority_queue 小根堆是否写了 greater<>。
- mod 运算后是否可能为负，必要时 $(x \% MOD + MOD) \% MOD$ 。
- 递归深度是否可能爆栈，树/图深 DFS 可改迭代或确认环境。
- 输出格式是否有多余空格、换行、精度、大小写。
- 本地样例、最小边界、最大边界、随机小数据对拍至少过一种。

23. 来源与规则参考

- ICPC World Finals Programming Environment 说明：<https://docs.icpc.global/worldfinals-programming-environment>
- ICPC 区域赛规则示例：<https://nc.na.icpc.global/rules/>。各赛区规则会单独发布，常见为纸质或 PDF Team Reference Document，但页数、纸张、单双面、是否可手写标注都要看当站规则。
- CCPC 第六届总决赛参赛指南：<https://final.ccpc.io/rules/>
- 蓝桥杯官网：<https://dasai.lanqiao.cn/>
- 蓝桥杯个人赛规则入口示例：https://dasai.lanqiao.cn/pages/v7/dasai/competition/competition_info.html?c=11&m=35
- 蓝桥杯个人赛管理办法公开转载：<https://www.newunivs.com/2907.html>
- 蓝桥杯第十五届个人线上比赛手册示例：<https://x-static.lanqiao.cn/upload/202405/courseimg/f123187c-8e16-445a-8cd3-9d73689e8901.pdf>
- 团体程序设计天梯赛竞赛规程：<https://gplt.patest.cn/regulation>

